

Memorial Técnico

1. Identificação do projeto

Projeto: Portal de Solicitações Internas

Desenvolvedor: Luiz Inácio Moura da Costa

O projeto foi desenvolvido como parte de um desafio técnico para uma vaga de Desenvolvedor Full Stack Júnior.

O objetivo foi criar uma aplicação web para registrar e acompanhar solicitações internas de uma empresa, permitindo autenticação de usuários, criação e gerenciamento de solicitações, filtros e visualização de indicadores.

2. Tecnologias utilizadas

Linguagem

- JavaScript

Frontend

- React
- Vite
- React Router DOM
- Axios
- CSS

Backend

- Node.js
- Express
- JSON Web Token (JWT)
- bcryptjs
- CORS
- dotenv
- sqlite e sqlite3

Banco de dados

- SQLite

Ferramentas de desenvolvimento

- npm
- Nodemon
- Git
- GitHub

3. Justificativa das tecnologias

JavaScript

O JavaScript foi utilizado tanto no frontend quanto no backend. Isso permitiu trabalhar com a mesma linguagem nas duas partes da aplicação, diminuindo a troca de contexto durante o desenvolvimento e facilitando a integração entre elas.

Para um projeto com prazo curto, essa escolha ajudou na produtividade e deixou a manutenção mais simples, já que a maior parte da aplicação segue a mesma base de linguagem.

React

O React foi escolhido para desenvolver a interface por permitir a criação da aplicação a partir de componentes reutilizáveis.

Também facilitou o gerenciamento dos dados exibidos nas páginas utilizando recursos como `useState` e `useEffect`.

Em comparação com uma interface feita apenas com JavaScript e manipulação direta do DOM, a separação em componentes facilita a organização e a manutenção das telas. Para o tamanho deste projeto, também permitiu reaproveitar elementos como o cabeçalho e a rota protegida sem repetir código.

Vite

O Vite foi utilizado para criar e executar o projeto React por possuir uma configuração simples e um ambiente de desenvolvimento rápido.

Em comparação com uma configuração manual das ferramentas de build, o Vite reduziu a quantidade de configuração necessária e agilizou o desenvolvimento. Isso foi útil principalmente pelo prazo reduzido do desafio.

React Router DOM

Foi utilizado para controlar a navegação entre as páginas da aplicação, como login, dashboard, listagem, criação, detalhes e edição de solicitações.

Também foi utilizado na criação de rotas protegidas para evitar que usuários não autenticados acessem diretamente as páginas internas.

Sem uma biblioteca de roteamento, esse controle teria que ser implementado manualmente. O React Router deixou a navegação centralizada e mais fácil de manter conforme novas páginas fossem adicionadas.

Axios

O Axios foi escolhido para realizar a comunicação entre o frontend e a API do backend.

Com ele são realizadas as requisições de login, criação, consulta, edição, exclusão e alteração de status das solicitações.

A API poderia ser consumida utilizando o `fetch` nativo do navegador, mas o Axios permitiu centralizar a configuração do endereço da API e do cabeçalho de autenticação. Isso reduziu repetição de código nas páginas do frontend.

CSS

O CSS foi utilizado diretamente nos componentes e páginas para construir o layout e os ajustes de responsividade.

Para o escopo do desafio, foi preferido em vez de adicionar uma biblioteca de componentes ou framework de estilos. Dessa forma, a interface continuou simples e o projeto ganhou menos dependências externas.

Node.js e Express

O backend foi desenvolvido com Node.js e Express.

O Express foi utilizado para criar as rotas da API e organizar o tratamento das requisições HTTP.

A escolha foi feita principalmente pela simplicidade de integração com o frontend em JavaScript e pela facilidade de criar uma API REST. Em comparação com uma solução que utilizasse outra linguagem no backend, manter JavaScript nas duas partes ajudou na produtividade durante o desenvolvimento.

A separação entre rotas, controladores, middlewares e acesso ao banco também permite que novas funcionalidades sejam adicionadas sem concentrar toda a lógica em um único arquivo.

SQLite, sqlite e sqlite3

O SQLite foi escolhido por ser um banco de dados simples e adequado ao tamanho do projeto.

Ele não exige a instalação de um servidor separado e permite que o banco seja criado automaticamente pela própria aplicação. Para um projeto de pequeno porte e execução local, isso facilitou a configuração e os testes.

Em comparação com PostgreSQL ou MySQL, o SQLite possui uma preparação inicial menor. Por outro lado, bancos cliente-servidor seriam mais adequados em um cenário de produção com maior quantidade de usuários simultâneos.

As bibliotecas `sqlite` e `sqlite3` são utilizadas pelo backend para abrir o arquivo do banco, executar o script de criação das tabelas e realizar as consultas necessárias.

JWT

O JSON Web Token é utilizado para controlar a autenticação do usuário.

Depois de realizar o login com sucesso, o backend gera um token que deve ser enviado nas requisições das rotas protegidas.

A opção por JWT evitou a necessidade de manter sessões no servidor para este projeto. Para uma aplicação pequena, isso deixou o fluxo de autenticação simples. Em uma aplicação de produção, a forma de armazenamento e envio do token precisaria receber cuidados adicionais de segurança.

bcryptjs

O `bcryptjs` é utilizado para gerar o hash da senha do usuário.

Dessa forma, a senha não é armazenada diretamente em texto puro no banco de dados. Em comparação com armazenar a senha diretamente ou utilizar apenas um hash simples, o `bcrypt` foi escolhido por ser próprio para armazenamento de senhas e utilizar um custo configurável no processo de hash.

CORS

O CORS foi utilizado no backend para permitir a comunicação entre o frontend e a API durante o desenvolvimento, já que eles são executados em portas diferentes.

No ambiente local foi utilizada uma configuração simples. Em produção, o ideal seria restringir as origens permitidas de acordo com o endereço real do frontend.

dotenv

O dotenv foi utilizado para carregar configurações através de variáveis de ambiente, como a chave utilizada na geração dos tokens JWT e a porta do servidor.

Isso evita deixar valores de configuração diretamente no código e facilita a utilização de configurações diferentes entre desenvolvimento e produção.

npm e Nodemon

O npm é utilizado para instalar e gerenciar as dependências do frontend e do backend.

O Nodemon é utilizado apenas durante o desenvolvimento para reiniciar o backend automaticamente quando o código é alterado. Isso não muda o funcionamento final da aplicação, mas melhora a produtividade durante a implementação.

Git e GitHub

O Git foi utilizado para versionamento do projeto e o GitHub para armazenamento do repositório e entrega do código.

O versionamento permite acompanhar as alterações realizadas e mantém o projeto organizado durante o desenvolvimento.

4. Organização da aplicação

O projeto foi separado principalmente em frontend, backend, banco de dados e documentação.

```
portal-solicitacoes/  
├── backend/  
├── database/  
├── docs/  
├── frontend/  
└── README.md
```

Essa separação foi utilizada para deixar mais claro o papel de cada parte do sistema.

No backend foi adotada uma separação simples de responsabilidades entre rotas, controladores, middlewares e acesso ao banco. Não foi utilizado um padrão arquitetural complexo, pois o projeto possui escopo reduzido. A intenção foi manter a estrutura fácil de entender e permitir que cada parte tivesse uma responsabilidade bem definida.

No frontend, as telas foram separadas em páginas, os elementos reutilizáveis ficaram em componentes e a comunicação com a API foi concentrada na pasta de serviços.

5. Organização do backend

O backend está organizado nas seguintes pastas:

```
backend/src/  
├─ banco/  
├─ controladores/  
├─ middlewares/  
├─ rotas/  
└─ servidor.js
```

banco

Contém os arquivos responsáveis pela conexão com o SQLite e pela criação do usuário inicial utilizado para demonstração.

controladores

Contém a lógica utilizada pelas rotas.

Foram separados controladores para:

- autenticação;
- solicitações;
- indicadores do dashboard.

middlewares

Contém o middleware responsável pela verificação do token JWT.

Antes de permitir o acesso às rotas protegidas, o middleware verifica se o token enviado é válido.

rotas

Contém a definição dos endpoints da API.

As rotas foram separadas de acordo com a funcionalidade:

- autenticação;
- solicitações;
- dashboard.

servidor.js

É o ponto de inicialização do backend.

Nesse arquivo são configurados o Express, CORS, leitura de JSON, conexão com o banco e registro das rotas da aplicação.

6. Organização do frontend

O frontend está organizado principalmente em:

```
frontend/src/  
├─ componentes/  
├─ paginas/  
├─ servicos/  
├─ App.jsx  
└─ main.jsx
```

componentes

Contém componentes utilizados em diferentes partes da aplicação.

Entre eles estão o cabeçalho e o componente de rota protegida.

paginas

Contém as principais telas do sistema:

- Login;
- Dashboard;
- Solicitações;
- Nova Solicitação;
- Detalhes da Solicitação;
- Editar Solicitação.

servicos

Contém funcionalidades utilizadas por diferentes páginas.

Nesse local fica a configuração do Axios para comunicação com a API e a função utilizada para formatação das datas.

App.jsx

Contém a definição das rotas da aplicação.

As páginas internas são protegidas pelo componente RotaProtegida.

7. Modelagem do banco de dados

O banco possui duas tabelas principais:

- usuarios;
- solicitacoes.

A tabela usuarios armazena os dados necessários para autenticação.

A tabela solicitacoes armazena os dados das solicitações criadas no sistema.

Existe um relacionamento entre as duas tabelas através do campo:

solicitacoes.usuario_id

que referencia:

usuarios.id

Dessa forma, cada solicitação fica relacionada ao usuário que realizou seu cadastro.

A modelagem foi mantida simples porque o sistema possui poucas entidades. As categorias e os status foram tratados como valores controlados pela aplicação, sem a criação de tabelas separadas para eles. Para este escopo isso reduz a complexidade do banco. Em uma evolução com categorias configuráveis ou regras mais complexas, essa modelagem poderia ser revista.

Os detalhes dos campos estão disponíveis no arquivo:

docs/dicionario-de-dados.md

8. Autenticação

O acesso às funcionalidades internas depende de autenticação.

O processo funciona da seguinte forma:

1. O usuário informa seu usuário e senha na tela de login.
2. O frontend envia os dados para o backend.
3. O backend procura o usuário no banco.
4. A senha informada é comparada com o hash armazenado utilizando bcrypt.
5. Caso os dados estejam corretos, o backend gera um token JWT.
6. O frontend armazena o token e o envia nas próximas requisições.
7. O middleware do backend verifica o token antes de permitir o acesso às rotas protegidas.

O token possui tempo de expiração configurado no backend.

O frontend também utiliza uma rota protegida para impedir a navegação normal para páginas internas quando não existe um token armazenado.

A validação realizada pelo backend é a responsável pela proteção efetiva da API.

9. Comunicação entre frontend e backend

A comunicação é realizada através de uma API HTTP.

O frontend utiliza Axios para fazer as requisições para a API. No ambiente local, o endereço padrão é:

`http://localhost:3000/api`

Esse endereço também pode ser configurado pela variável de ambiente `VITE_API_URL`, evitando a necessidade de alterar o código caso frontend e backend sejam publicados em endereços diferentes.

Entre as principais operações estão:

```
POST /api/autenticacao/login
GET /api/solicitacoes
POST /api/solicitacoes
GET /api/solicitacoes/:id
PUT /api/solicitacoes/:id
DELETE /api/solicitacoes/:id
PATCH /api/solicitacoes/:id/status
GET /api/dashboard
```

As rotas de solicitações e dashboard exigem autenticação.

10. Regras das solicitações

Uma nova solicitação possui:

- título;
- descrição;
- categoria;
- usuário responsável pela criação;
- data de criação automática;

- status inicial Aberto.

As categorias disponíveis são:

- TI;
- RH;
- Compras;
- Financeiro;
- Infraestrutura.

Os status utilizados são:

- Aberto;
- Em Atendimento;
- Concluído.

Solicitações com status Aberto podem ser editadas ou excluídas.

A alteração de status é feita separadamente através de uma rota específica.

11. Filtros

A listagem de solicitações permite a utilização de filtros por:

- título;
- categoria;
- status;
- período.

Os filtros são enviados pelo frontend através dos parâmetros da requisição.

No backend, a consulta SQL é montada de acordo com os filtros recebidos.

Foram utilizados parâmetros na consulta para evitar a inserção direta dos valores enviados pelo usuário no SQL.

12. Dashboard

O dashboard apresenta quatro indicadores:

- total de solicitações;
- solicitações abertas;
- solicitações em atendimento;
- solicitações concluídas.

Os valores são calculados diretamente no banco de dados e retornados pelo backend.

13. Tratamento de erros e validações

Foram adicionadas validações tanto no frontend quanto no backend.

O frontend utiliza campos obrigatórios nos formulários e exibe mensagens quando uma operação não pode ser concluída.

O backend também verifica os dados recebidos antes de realizar alterações no banco.

Alguns exemplos são:

- usuário ou senha não informados;
- categoria inválida;

- status inválido;
- solicitação inexistente;
- tentativa de editar uma solicitação que não está aberta;
- tentativa de excluir uma solicitação que não está aberta;
- acesso a uma rota protegida sem um token válido.

As validações do backend são mantidas mesmo quando existem validações equivalentes no frontend, pois as requisições para a API também podem ser realizadas sem utilizar a interface.

14. Responsividade

A interface recebeu ajustes para funcionar em diferentes tamanhos de tela.

No dashboard, a quantidade de colunas dos indicadores é reduzida conforme a largura disponível.

Os formulários e botões também se adaptam a telas menores.

Na listagem de solicitações, a tabela utiliza rolagem horizontal quando não existe espaço suficiente para mostrar todas as colunas.

15. Limitações atuais

Por se tratar de um projeto desenvolvido para um desafio técnico e com escopo reduzido, algumas funcionalidades não foram implementadas.

Entre as principais limitações estão:

- não existe tela para cadastro de novos usuários;
- o usuário de demonstração é criado através de um script;
- não existem diferentes níveis ou perfis de acesso;
- não existe paginação na listagem de solicitações;
- não foram implementados testes automatizados;
- o banco SQLite é local;
- não existe implantação em ambiente de produção.

Atualmente, as operações são autorizadas para usuários autenticados, enquanto as regras de edição e exclusão são controladas principalmente pelo status da solicitação.

16. Melhorias futuras

Como evolução do sistema, poderiam ser adicionados:

- cadastro e gerenciamento de usuários;
- diferentes perfis de acesso;
- restrição de edição e exclusão de acordo com o usuário responsável;
- paginação da listagem;
- histórico das alterações de status;
- testes automatizados de frontend e backend;
- recuperação de senha;
- logs de operações;
- notificações;
- implantação da aplicação em ambiente de produção.

17. Diferenças para um ambiente de produção

Em um ambiente real de produção, algumas decisões técnicas seriam revistas.

O SQLite poderia ser substituído por um banco de dados como PostgreSQL ou MySQL, principalmente em um cenário com vários usuários acessando o sistema ao mesmo tempo.

A aplicação também deveria utilizar HTTPS e uma configuração mais segura para armazenamento e envio do token de autenticação.

A chave utilizada para gerar os tokens JWT deve ser configurada através de uma variável de ambiente segura e possuir um valor forte. O projeto já utiliza arquivo de exemplo para indicar as variáveis necessárias, sem exigir que o arquivo .env real seja versionado.

Também seriam importantes recursos como:

- controle de permissões;
- logs;
- monitoramento;
- tratamento centralizado de erros;
- testes automatizados;
- proteção contra excesso de requisições;
- configuração específica de CORS;
- processo automatizado de deploy.

18. Conclusão

O projeto atende às principais funcionalidades propostas para o Portal de Solicitações Internas.

Durante o desenvolvimento foi possível trabalhar com a integração entre frontend, backend e banco de dados, além de autenticação, rotas protegidas, operações CRUD, filtros e construção de uma interface responsiva.

A solução foi mantida simples, priorizando a funcionalidade, organização do código e facilidade de execução do projeto.